AI Coding Workshop — Matt Pocock

Conference · 2025 · 2 hours

> *Software engineering fundamentals that work with humans also work exceptionally well with AI — the key is understanding LLM constraints and applying proven practices like TDD, modular design, and human-in-the-loop alignment.*

LLM Constraints: Smart Zone vs Dumb Zone

- LLMs have a ‘smart zone’ (early context, 0-100K tokens) where they perform best, and a ‘dumb zone’ (beyond 100K) where performance degrades due to quadratic attention relationships between tokens
- Size tasks to fit within the smart zone rather than letting context bloat into the dumb zone through continuous conversation
- Break large tasks into small, parallelizable chunks that each stay in the smart zone, rather than using multi-phase sequential plans
- LLMs are like the protagonist from Memento — they reset to base state when context clears, so optimize for small system prompts and repeatable sessions

Every time you add a token to an LLM, it’s like adding a team to a football league — the number of matches scales quadratically.

From Alignment to Implementation: PRDs and Kanban

- Convert the grilling session into two documents: a PRD (destination) and a Kanban board (journey)
- The PRD describes the end state — what success looks like — without over-specifying implementation details
- The Kanban board breaks work into small, parallelizable issues with explicit blocking relationships
- Use sub-agents for exploration (isolated context windows that report summaries back) to preserve main context budget
- Don’t over-optimize the PRD — alignment happens in grilling, not in perfecting documentation

Deep Modules: Architecture for AI Success

- Shallow modules (many small files with complex dependencies) confuse AI and make testing boundaries unclear
- Deep modules (small interface, lots of internal functionality) are easier for AI to navigate and test as cohesive units
- Design module interfaces yourself, then delegate implementation to AI — this preserves your understanding of the codebase while moving fast
- Run the ‘improve codebase architecture’ skill to identify clusters of related code that should be consolidated into deep modules
- Without good feedback loops and testable architecture, AI will produce poor code regardless of prompting technique

If your codebase doesn’t have feedback loops, you’re never going to get decent output from AI. The quality of your feedback loops is the ceiling.

Practical Workflow Tips

- Monitor token usage constantly (add a status line showing exact token count) to know how close you are to the dumb zone
- Use dedicated tools (file readers, editors, search) instead of terminal commands when available for better visibility
- Prefer clearing context over compacting — compacting creates unreliable ‘sediment’ that degrades over time
- For parallelization, use tools like Sandcastle (TypeScript library) to run multiple agents in isolated Docker containers with separate Git branches
- Treat AI like pair programming with a third person who relentlessly quizzes you — bring domain experts into grilling sessions

We all think AI is a new paradigm, but software engineering fundamentals that work with humans also work super well with AI.

The Grill Me Skill: Achieving Alignment

- Start every project with a ‘grill me’ session where the AI interviews you relentlessly about requirements until reaching shared understanding (the ‘design concept’)
- This prevents the biggest AI failure mode: misalignment between what you want and what gets built
- The AI provides recommended answers to its own questions, often surfacing good defaults you hadn’t considered
- Grilling sessions can ask 40-100 questions and last an hour — this is human-in-the-loop work that cannot be automated
- The conversation history becomes an asset documenting the design concept, useful for onboarding or revisiting decisions

We need to reach a shared understanding. I need an asset, I didn’t need a plan, I needed to be on the same wavelength as the AI.

The RALF Loop: Autonomous Implementation

- RALF (inspired by Ralph Wiggum) is a loop pattern: specify the destination, then repeatedly make small changes that get closer to it
- Each iteration stays in the smart zone by clearing context between cycles rather than compacting (which creates ‘sediment’ and drift)
- Use TDD (test-driven development) religiously — write failing test first, implement to pass, refactor — to prevent AI from cheating tests
- Feedback loops (tests, type checking, linting) are the ceiling for AI code quality — improve your feedback loops to improve AI output
- Implementation can be AFK (away from keyboard) work, but planning and QA must be human-in-the-loop

If you find an approach has failed twice, diagnose the root cause rather than making incremental patches.

Code Review and Quality Control

- Always clear context before reviewing — reviewing in the smart zone catches more issues than reviewing in the dumb zone after implementation
- QA manually to impose taste and catch issues automated testing misses — this is where human judgment matters most
- Use push (always-on instructions) for reviewers and pull (optional skills) for implementers when enforcing coding standards
- Delete or close PRDs after implementation to prevent doc rot — outdated documentation misleads AI more than missing documentation
- Parallelize implementation across multiple sandboxed branches, then merge with a dedicated merger agent that resolves conflicts